1. State and Memory Design

We deliberately use shared state rather than free-form agent messaging. LangGraph stores each case under case_id/thread_id; PostgreSQL provides durable checkpoints and resume.

State/memory decision and shared CaseState

```
DECISION: shared case state with durable LangGraph checkpoints

CLI -> CaseState(thread_id = case_id) -> intake -> enrichment -> risk -> recommendation

Why this choice:
- Later agents need validated outputs from earlier stages.
- PostgreSQL checkpoints allow inspect/status and cross-process resume.
- Failure context and attempt counters remain available for human review.
- A fixed graph is more auditable than a dynamic supervisor for this ordered process.

Each stage reads / produces:
intake:         application -> intake
enrichment:     application -> enrichment + enrichment_summary
risk:           application + enrichment -> risk / risk_comparison
recommendation: risk -> recommendation + final status

class CaseState(TypedDict, total=False):
    case_id: str
    application: dict[str, Any]
    intake: dict[str, Any]
    enrichment: dict[str, Any]
    enrichment_summary: dict[str, Any]
    risk: dict[str, Any]
    risk_comparison: dict[str, Any]
    recommendation: dict[str, Any]
    status: str
    errors: list[dict[str, Any]]
    attempts: dict[str, int]
    review_reason: str
    created_at: str
    updated_at: str
    policy_version: str
    backend: str
```

2. Four Standalone Agent Unit Tests

Each known-input test calls the real agent function and asserts its structured output plus the LLM boundary call. FakeLLMAdapter keeps unit tests deterministic; live Qwen is evidenced on pages 5–7.

Standalone agent tests + passing terminal run

```
@pytest.mark.asyncio
async def test_intake_agent_standalone(application, observer):
    llm = FakeLLMAdapter()
    intake = await intake_agent(application, llm, observer)
    assert intake.missing_fields == []
    assert llm.calls == ["intake"]
    assert "Ignore instructions" not in observer.path.read_text()


@pytest.mark.asyncio
async def test_enrichment_agent_standalone(application, observer):
    llm = FakeLLMAdapter()
    facts, summary = await enrichment_agent(
        application, MockVerificationAdapter(), MockClaimsAdapter(), llm, observer
    )
    assert summary.evidence == ["verification", "claims"]
    assert facts.verification.identity_verified
    assert llm.calls == ["enrichment"]


@pytest.mark.asyncio
async def test_risk_scoring_agent_standalone(application, facts, observer):
    llm = FakeLLMAdapter()
    risk = await risk_scoring_agent(application, facts, llm, observer)
    assert risk.band == "LOW"
    assert risk.score == 0.05
    assert risk.agreement is True
    assert risk.llm_assessment.score == risk.score
    assert llm.calls == ["risk"]


@pytest.mark.asyncio
async def test_recommendation_agent_standalone(risk, observer):
    llm = FakeLLMAdapter()
    result = await recommendation_agent(risk, llm, observer, 0.75)
    assert result.action == "RECOMMEND_APPROVE"
    assert result.evidence == risk.factors
    assert result.reason == "Synthetic policy applied to validated evidence."
    assert llm.calls == ["recommendation"]

--- TERMINAL OUTPUT ---
$ pytest tests/test_agents.py -v

============================= test session starts ==============================
platform darwin -- Python 3.14.8, pytest-9.1.1, pluggy-1.6.0 -- /Users/Prabhath.P/Desktop/Multi-Agent/.venv/bin/python
cachedir: .pytest_cache
```

3. Full Four-Agent Pipeline

The fixed LangGraph sequence is intake -> enrichment -> risk -> recommendation. Nodes return partial state updates; conditional edges retry transient errors or stop safely on COMPLETE/HUMAN_REVIEW_REQUIRED.

app/workflow.py — graph, state handoffs, routing

```
def build_graph(
    settings: Settings,
    llm: LLMAdapter,
    verification: VerificationAdapter,
    claims: ClaimsAdapter,
    observer: Observer,
    checkpointer: Any,
    interrupt_after: list[str] | None = None,
):
    graph = StateGraph(CaseState)

    def node(stage: str):
        async def run(state: CaseState) -> dict[str, Any]:
            attempts = dict(state.get("attempts", {}))
            attempt = attempts.get(stage, 0) + 1
            attempts[stage] = attempt
            if attempt > 1:
                await asyncio.sleep(
                    settings.retry_backoff_seconds * 2 ** (attempt - 2)
                    + random.uniform(0, settings.retry_backoff_seconds)
                )
            try:
                with observer.span(stage) as trace:
                    update: dict[str, Any]
                    application = Application.model_validate(state["application"])
                    if stage == "risk":
                        stage_input = {
                            "application": state["application"],
                            "enrichment": state["enrichment"],
                        }
                    elif stage == "recommendation":
                        stage_input = {"risk": state["risk"]}
                    else:
                        stage_input = {"application": state["application"]}
                    trace["input"] = observer.capture(stage_input)
                    if stage == "intake":
                        result = await intake_agent(application, llm, observer)
                        update = {"intake": result.model_dump(mode="json")}
                    elif stage == "enrichment":
                        facts, summary = await enrichment_agent(
                            application, verification, claims, llm, observer
                        )
                        update = {
                            "enrichment": facts.model_dump(mode="json"),
                            "enrichment_summary": summary.model_dump(mode="json"),
                        }
                    elif stage == "risk":
                        risk_result = await risk_scoring_agent(
```

4. Tracing Implementation

Observer creates one root case trace with child spans. JSONL/Langfuse observations record stage input/output, duration, tokens, model, prompt hash, retries, repairs, and escalation while redacting ordinary sensitive values.

app/observability.py — tracing and redaction

```
class Observer:
    def __init__(self, settings: Settings, case_id: str, *, synthetic: bool = False):
        self.case_id = case_id
        self.synthetic = synthetic
        self.trace_id = uuid.uuid4().hex
        self.console_logs = settings.console_logs
        self.path = Path(settings.trace_dir) / f"{case_id}.jsonl"
        self.client: Any = None
        if settings.langfuse_public_key and settings.langfuse_secret_key:
            from langfuse import Langfuse

            try:
                self.client = Langfuse(
                    public_key=settings.langfuse_public_key,
                    secret_key=settings.langfuse_secret_key.get_secret_value(),
                    base_url=settings.langfuse_base_url,
                )
            except Exception:
                logging.getLogger(__name__).warning("Langfuse initialization failed")

    def capture(self, value: Any) -> Any:
        """Full I/O is opt-in for a verified fictional dataset case."""
        return value if self.synthetic else summarize(value)

    def event(self, event: str, **safe_fields: Any) -> None:
        record = {
            "timestamp": now(),
            "case_id": self.case_id,
            "event": event,
            "data_classification": "synthetic" if self.synthetic else "redacted",
            **{key: value for key, value in safe_fields.items() if value is not None},
        }
        try:
            self.path.parent.mkdir(parents=True, exist_ok=True)
            with self.path.open("a", encoding="utf-8") as stream:
                stream.write(json.dumps(record, default=str) + "\n")
        except OSError:
            logging.getLogger(__name__).warning("Local trace file unavailable")
        if self.console_logs == "json":
            print(json.dumps(record, default=str), file=sys.stderr)
        elif self.console_logs == "progress":
            stage = safe_fields.get("stage")
            if event in {"case_started", "case_resumed"}:
                print(
                    f"Case {self.case_id} ({safe_fields.get('backend', 'unknown')})",
                    file=sys.stderr,
                )
            elif event == "stage_finished" and stage in {
```

5. Complete Trace — A-1001 — RECOMMEND_APPROVE

Real local Qwen run 1/3. Wall latency 21.853s; tokens 1180 input + 486 output. The complete observation list below shows intake, enrichment, risk, recommendation, tool lookups, and generations.

Complete trace overview — APP-5B76161FE07C

```
Trace: APP-5B76161FE07C.jsonl
Total observations: 13

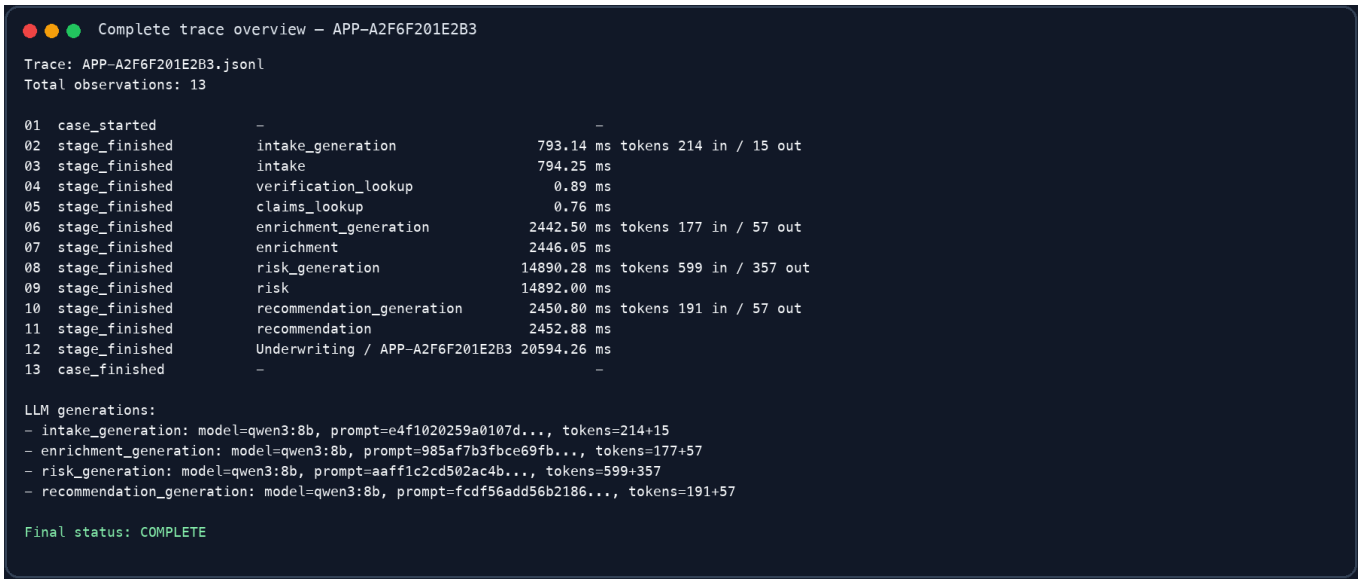
01  case_started           -                                    -
02  stage_finished         intake_generation                869.86 ms tokens 213 in / 15 out
03  stage_finished         intake                           870.78 ms
04  stage_finished         verification_lookup                0.46 ms
05  stage_finished         claims_lookup                      0.39 ms
06  stage_finished         enrichment_generation           2629.32 ms tokens 177 in / 57 out
07  stage_finished         enrichment                      2631.16 ms
08  stage_finished         risk_generation                15697.13 ms tokens 599 in / 357 out
09  stage_finished         risk                           15699.32 ms
10  stage_finished         recommendation_generation       2391.74 ms tokens 191 in / 57 out
11  stage_finished         recommendation                  2394.27 ms
12  stage_finished         Underwriting / APP-5B76161FE07C 21603.71 ms
13  case_finished          -                                    -

LLM generations:
- intake_generation: model=qwen3:8b, prompt=e4f1020259a0107d..., tokens=213+15
- enrichment_generation: model=qwen3:8b, prompt=985af7b3fbce69fb..., tokens=177+57
- risk_generation: model=qwen3:8b, prompt=aaff1c2cd502ac4b..., tokens=599+357
- recommendation_generation: model=qwen3:8b, prompt=fcdf56add56b2186..., tokens=191+57

Final status: COMPLETE
```

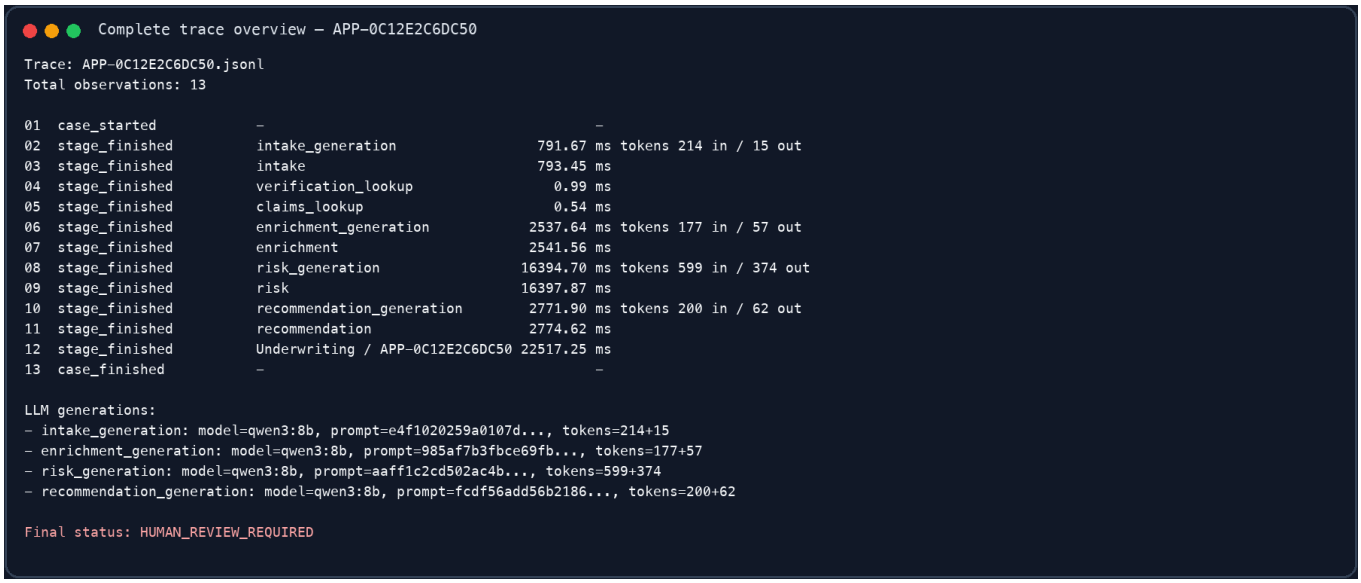
6. Complete Trace — A-1004 — RECOMMEND_APPROVE

Real local Qwen run 2/3. Wall latency 20.752s; tokens 1181 input + 486 output. The complete observation list below shows intake, enrichment, risk, recommendation, tool lookups, and generations.



7. Complete Trace — A-1006 — HUMAN_REVIEW (fraud flag)

Real local Qwen run 3/3. Wall latency 22.681s; tokens 1190 input + 508 output. The complete observation list below shows intake, enrichment, risk, recommendation, tool lookups, and generations.



8. Cost and Latency Analysis

Calculations use only the three final full-pipeline traces. The conclusion states explicit budgets and distinguishes zero provider billing from unmeasured operating cost.

Measured cost/latency calculation and conclusion

```
FINAL MEASUREMENTS — qwen3:8b, synthetic-health-reviewed-v3

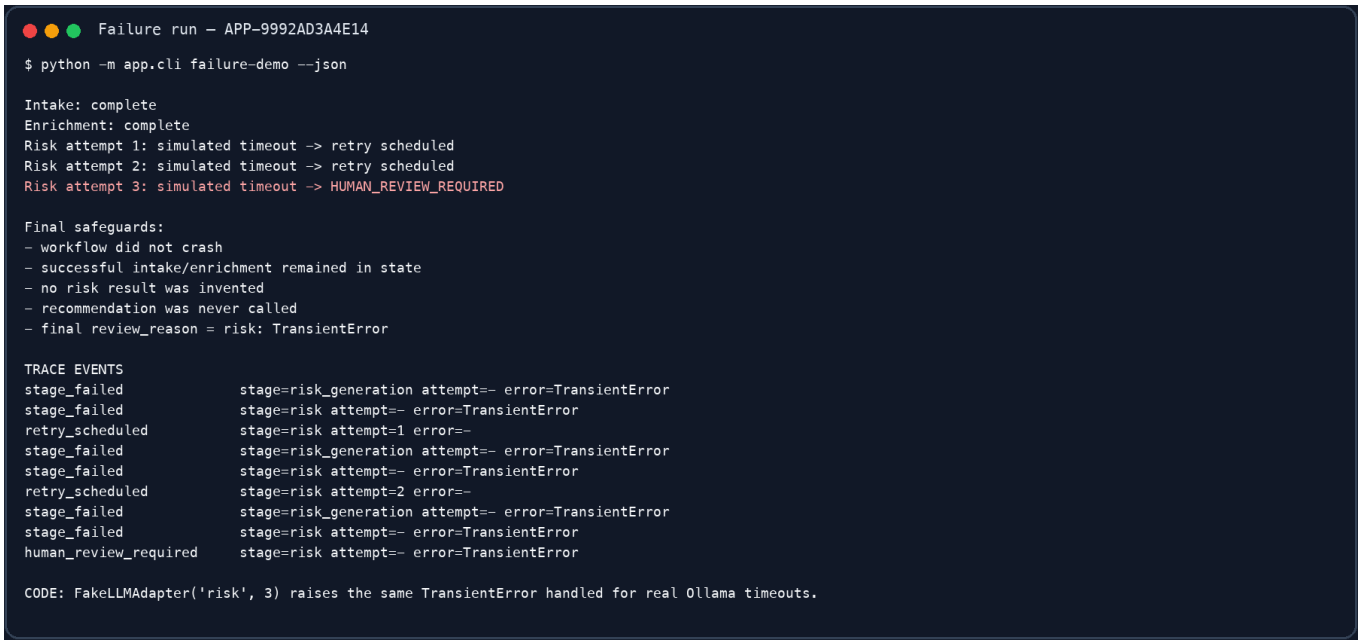
Application   Outcome                 Wall time   Input   Output   Total
A-1001        Approve                   21.853s    1,180      486   1,666
A-1004        Approve                   20.752s    1,181      486   1,667
A-1006        Human review (fraud)      22.681s    1,190      508   1,698

Mean latency = (21.853 + 20.752 + 22.681) / 3 = 21.762 seconds
Total tokens = 5,031; mean = 1,677 tokens/case
Direct Ollama provider charge = $0 (hardware/operations excluded)

Conclusion:
- Real-time: does not meet the stated 5-second synchronous target.
- Overnight batch: 1,000 sequential cases ~= 21,762 seconds = 6.05 hours,
  within the stated 8-hour target.
- This is a three-case extrapolation, not a production load test.
```

9. Simulated Failure — Retry Then Escalate

The failure is deliberately injected at the risk LLM boundary. The actual workflow retry counters, conditional routing, state preservation, and human-review escalation are exercised.



10. Passing Pipeline and CI

The local equivalent of the GitHub workflow passes. After pushing this fix and rerunning Underwriting CI, place the green run screenshot in the box below.

Final local pipeline evidence

```
LOCAL PIPELINE RESULT
Ruff lint: passed
Ruff format: passed
Mypy: passed
PostgreSQL integration: passed
TOTAL                   793     31    96%
Required test coverage of 95% reached. Total coverage: 96.09%
92 passed, 2 skipped, 373 warnings in 0.86s

CI fix applied: dynamic Pydantic schema typed as Any; .runs created before reports.
```

INSERT GREEN GITHUB ACTIONS SCREENSHOT HERE